

Matlala L.B. (231515216)

Role: User Management Module Developer and Database Administrator

Introduction

The INT316D module, Internet Programming, has been one of the most challenging and valuable modules I have completed this year. Before starting this course, I had limited knowledge of enterprise web development and had never built a complete Java web application. Throughout the semester, I learned important concepts such as the evolution of Java Enterprise Edition (JEE), the Model-View-Controller (MVC) architecture, JSP and Servlets, exception handling, Enterprise JavaBeans (EJB), Java Persistence API (JPA), JPQL, and application security. These topics helped me understand how professional enterprise systems are designed and maintained.

For our group project, we developed SmartFlow, a real-time traffic management system designed for the Witbank area. The purpose of the system was to improve traffic monitoring and management by allowing different users to manage incidents, sensors, and traffic signals through one centralized application. My role in the project was both Database Administrator and User Management Module Developer. I was responsible for designing the database structure and building the module that allows administrators to manage user accounts securely. This reflection explains my contributions to the project and how I applied the concepts learned in INT316D.

My Role in the SmartFlow Project

My first responsibility was designing the SmartFlow database. I created the tables required for the system, including users, traffic_sensors, traffic_logs, traffic_incidents, and traffic_signals. I defined primary keys, foreign keys, and constraints to maintain data integrity and ensure relationships between tables were correct. I also added sample data to each table so that the system could be tested properly.

In addition, I created stored procedures used by other modules in the project. One procedure handled incident resolution updates, while another inserted a new traffic sensor together with its initial log entry. These procedures helped ensure database consistency because related operations were completed together as one transaction.

My second responsibility was developing the User Management Module. This module allows administrators to perform CRUD operations, which include creating, reading, updating, and deleting user accounts. I created a page that displays all users in a table with information such as username, email address, role, and account creation date. Administrators can add new users using a form, update user roles using dropdown menus, and remove users with a delete button.

I also implemented several additional controls to improve the interface and satisfy project requirements. These included radio buttons for role selection, a date picker for account expiry dates, and toggle switches for features such as email notifications and two-factor authentication. I added a safety feature that prevents administrators from deleting their own accounts, which improves system security.

Application of INT316D Topics

Evolution of JEE

Learning about the evolution of JEE helped me understand why enterprise applications use layered architectures and frameworks. Earlier web technologies mixed business logic and presentation code, making systems difficult to maintain. Modern JEE separates responsibilities into different layers, improving scalability and maintainability. In SmartFlow, I used servlets as controllers, JSP pages as views, and Java classes as models. Understanding the history of JEE helped me appreciate why enterprise systems follow these design patterns.

Model-View-Controller (MVC)

The MVC architecture was one of the most important concepts applied in my module. The Model was represented by the User class and database tables. The View was the users.jsp page that displayed forms and tables for administrators. The Controller was the UserServlet, which handled requests and communicated with the database.

Using MVC made the application easier to manage because each layer had its own responsibility. The JSP page focused only on displaying data, while the servlet handled business logic and database communication. This separation improved readability and made debugging easier.

JSP and Servlets

JSP and Servlets were essential for building the User Management Module. I used JSP tags and Expression Language to display user information dynamically on the page. The servlet processed form submissions, validated user input, and handled add, update, and delete operations.

The doGet method was used to retrieve and display users, while the doPost method handled modifications to the database. I learned how servlets act as controllers that process requests and send responses back to the client. Working with JSP and Servlets helped me understand how Java web applications function from request to response.

Exception Handling

Exception handling was necessary to make the system reliable and user-friendly. Many problems can occur in a user management system, such as duplicate usernames, database connection failures, or invalid input. I used try-catch-finally blocks throughout my code to handle these situations.

For example, when a duplicate username is entered, the application displays a friendly error message instead of crashing. I also ensured that database connections and resources were always closed properly in the finally block. Learning exception handling taught me that professional applications must handle errors gracefully and continue running without exposing technical details to users.

Enterprise JavaBeans (EJB)

Although our project did not fully implement EJB, I learned how EJB simplifies enterprise application development. EJB provides services such as transaction management, security, and scalability. In a future version of SmartFlow, I would use a session bean to manage user operations.

For example, a UserService bean could handle account creation and validation while the container manages transactions automatically. This would reduce boilerplate code and improve

maintainability. Learning about EJB helped me understand how large enterprise systems support many users at the same time.

Java Persistence API (JPA) and JPQL

INT316D also introduced me to JPA and JPQL. Instead of manually writing JDBC code, JPA allows Java classes to map directly to database tables using annotations. This approach reduces code duplication and improves portability.

JPQL queries operate on Java objects rather than tables. For example, a query such as `SELECT u FROM User u WHERE u.role = 'Admin'` is easier to read and database-independent. Although we used JDBC in SmartFlow for simplicity, I now understand the advantages of JPA in enterprise development.

Application Security

Security was one of the most important parts of my module because user accounts control access to the entire system. I implemented login validation, role-based access control, and session management to protect sensitive pages.

Only administrators are allowed to access the User Management Module. I also used prepared statements to prevent SQL injection attacks and added checks to stop administrators from deleting their own accounts. These measures improved system security and demonstrated the importance of protecting enterprise applications from unauthorized access.

Challenges and Solutions

One major challenge was designing a database structure that supported all modules without duplication or conflicts. I solved this by carefully planning relationships between tables and using foreign keys to maintain consistency.

Another challenge was implementing secure access control. Initially, unauthorized users could still access restricted pages directly through URLs. I solved this problem by adding session validation and role-checking functions before allowing access.

I also struggled with designing toggle switches and custom controls because I had limited front-end experience. After researching CSS and JavaScript examples, I successfully created interactive controls that improved the appearance of the module.

What I Learned Throughout the Semester

INT316D taught me much more than programming syntax. I learned how enterprise systems are designed, how databases support web applications, and how security protects sensitive information. I also improved my problem-solving and teamwork skills.

Working on SmartFlow taught me the importance of planning before coding. Testing also became an important part of my development process because even small mistakes can affect the entire application. I learned to debug errors carefully and test all possible scenarios.

Conclusion

The SmartFlow project gave me practical experience in building an enterprise Java web application. My responsibilities as Database Administrator and User Management Module

Developer allowed me to apply concepts such as MVC architecture, JSP and Servlets, exception handling, database design, security, and stored procedures.

This project improved both my technical and teamwork skills and gave me confidence in developing enterprise systems. The knowledge and experience gained in INT316D will help me in future academic projects and my future career in software development.